SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA1001	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	T. Tarun	**Reg. No.:**	1924260001
Date:	26-10-2026	**Duration:**	60 minutes

Code of Conduct
I T. Tarun (1924260001) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: T. Tarun

Annexure A – Architecture Design Assignment

Instructions to Students:
Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Real-Time Water Management System

1. Introduction:

The Real-Time Water Management System is an IoT-based solution designed to monitor water levels, consumption, flow rate, and pressure in residential and institutional water networks. Sensors continuously collect measurements and help identify leakage, overflow, unusual consumption, and low water availability.

The proposed architecture integrates IoT sensors, data communication, real-time processing, alert management, secure storage, and a web dashboard. It reduces manual monitoring, supports timely maintenance, and helps authorities manage water resources efficiently.

2. System Requirements Analysis:

2.1 System Objectives

The main objectives of the Real-Time Water Management System are:

- To monitor water tank levels and water consumption continuously.
- To measure water flow and pressure using connected sensors.
- To detect leakage, tank overflow, and abnormal water usage.
- To provide real-time alerts to users and maintenance teams.
- To visualize water levels, consumption trends, and sensor status.
- To generate usage summaries and maintenance reports.
- To store reliable historical measurements and system activity logs.

2.2 Functional Requirements

Functional requirements describe the operations the system must perform.

ID	Requirement	Description
FR1	User Authentication	Allow authorized users to sign in securely.
FR2	Sensor Data Collection	Collect tank level, flow, and pressure readings.
FR3	Reading Validation	Validate sensor values and identify missing readings.
FR4	Leakage Detection	Detect unusual flow and possible pipeline leakage.
FR5	Overflow Monitoring	Identify high tank levels and overflow risks.
FR6	Dashboard Visualization	Display live levels, usage graphs, and device status.
FR7	Alert Management	Send notifications for leakage, overflow, or low water.
FR8	Report Generation	Generate water consumption and maintenance reports.
FR9	Activity Logging	Record device events, alerts, and user actions.

2.3 Non-Functional Requirements

Non-functional requirements describe the quality and performance expected from the system.

ID	Requirement	Description
NFR1	Scalability	Support more sensors and measurement records.
NFR2	Security	Protect user information and connected devices.
NFR3	Performance	Update dashboards and process alerts efficiently.
NFR4	Reliability	Handle device outages and preserve readings.

ID	Requirement	Description
NFR5	Maintainability	Allow modules and sensor integrations to be updated.
NFR6	Availability	Provide dependable monitoring access.
NFR7	Usability	Offer a clear interface for residents and managers.
NFR8	Data Integrity	Maintain accurate, timestamped sensor records.

2.4 Key Quality Attributes

The architecture must consider six important quality attributes:

- Scalability: Handles increasing numbers of sensors, buildings, and readings.
- Security: Protects user accounts, device communication, and water usage data.
- Performance: Processes sensor readings and sends alerts promptly.
- Reliability: Maintains dependable monitoring despite temporary device failures.
- Maintainability: Enables individual modules and sensors to be updated.
- Availability: Keeps monitoring and alerts accessible when needed.

3. Selection of Suitable Software Architecture

3.1 Selected Architecture: Hybrid Architecture

A Hybrid Architecture combining Layered Architecture and Microservices is suitable for the Real-Time Water Management System. It separates the dashboard, application logic, sensor processing, analytics, and database into clear layers, while allowing alerting and sensor ingestion to scale as separate services.

3.2 Architecture Components

The proposed architecture consists of the following layers:

Layer	Components	Purpose
Presentation Layer	Web Dashboard, Usage Charts	Shows live water status and notifications.
Application Layer	Authentication, Reports, Alert Settings	Manages user requests and application operations.
Sensor Processing Layer	Sensor Ingestion, Validation, Monitoring	Processes incoming IoT sensor measurements.

Layer	Components	Purpose
Analytics and Alert Layer	Threshold Rules, Usage Analysis, Notifications	Detects anomalies and issues timely alerts.
Data Layer	Database, Sensor History, Event Logs	Stores measurements and maintenance records.
External Interface Layer	IoT Sensors, Gateways, Notification APIs	Connects field devices and external services.

3.3 Justification for Selecting Hybrid Architecture

The hybrid approach is suitable for the following reasons:

1. Modularity: Separates sensor monitoring, analytics, alerts, and reporting.
2. Scalability: Allows sensor ingestion and alert processing to expand independently.
3. Maintainability: Enables updates to sensors, dashboards, or processing modules.
4. Security: Supports authenticated users, device validation, and secure APIs.
5. Performance: Processes continuous readings separately from dashboard requests.
6. Extensibility: Supports additional sensors, buildings, and prediction features.

3.4 Architecture Diagram

STAKEHOLDERS	SYSTEM ARCHITECTURE — DATA FLOW ↑	RESULTS
Residents • Water authorities • Maintenance team	① **IoT SENSOR NETWORK** **Water-level sensors | Flow meters | Pressure sensors | Leak sensors**	Live measurements
	▲ Sensor readings via Wi-Fi / LoRa / 4G ▲	
Device registration & access	② **COMMUNICATION / API GATEWAY** **Secure ingestion | Device validation | Message queue**	Connected devices
	▲ Validated sensor events ▲	
Administrator controls	③ **DATA PROCESSING SERVICES** **Data cleaning | Tank-level monitoring | Flow/pressure analysis | Leak detection**	Abnormality events
	▲ Processed metrics and events ▲	
	④ **ANALYTICS & DECISION ENGINE** **Usage trends | Demand forecasting | Threshold rules | Alert triggers**	Warnings & insights
	↕ Read / write ↕	
	⑤ **DATABASE & CLOUD STORAGE** **Time-series sensor readings | User accounts | Alert history | Reports**	Backup & audit logs
	↕ Secure APIs and notifications ↕	
Web / mobile users	⑥ **APPLICATION & PRESENTATION** **Authentication | Web dashboard | Mobile view | Reports | Alert delivery**	Live dashboard • SMS/email alerts • Reports

3.5 Architectural Trade-offs

A hybrid design adds service coordination, deployment effort, and communication overhead. Sensor failures and network interruptions also require buffering and recovery mechanisms. For an academic prototype, components can run as modules in one application before being separated into independent services.

4. Explanation of Components and Their Interactions

4.1 Component Description

The proposed architecture consists of the following components:

- Presentation Layer
  Provides a web dashboard for residents, facility managers, and administrators to view tank levels, consumption charts, sensor health, and alerts.
- Application Layer
  Handles user authentication, device registration, alert settings, water usage reports, and dashboard requests.
- Sensor Processing Layer
  Receives water level, flow, and pressure readings; validates measurements and identifies unusual conditions.
- Analytics and Alert Layer
  Analyzes usage trends, checks overflow and leakage thresholds, and generates notifications for responsible users.
- Data Layer
  Stores sensor measurements, tank information, user profiles, alert history, and maintenance records.
- External Interface Layer
  Connects IoT sensors, network gateways, notification services, and optional weather or utility APIs.

4.2 Interaction Between Components

Sensors transmit readings through a gateway to the Sensor Processing Layer. Validated data is saved in the Data Layer and evaluated by the Analytics and Alert Layer. The Application Layer provides

secure access to current measurements and reports, while the Presentation Layer displays status and notifications.

4.3 System Workflow

- The authorized user signs in to the monitoring dashboard.
- The administrator registers tanks, sensors, and alert thresholds.
- Water level, flow, and pressure sensors collect measurements.
- The communication gateway sends sensor readings to the backend.
- The processing module validates and timestamps the readings.
- The system stores measurements and evaluates leakage or overflow rules.
- The dashboard shows current tank levels and consumption trends.
- When abnormal conditions occur, the system notifies the responsible team.
- Maintenance actions and important events are recorded in the system log.

5. Discussion of Quality Attributes:

5.1 Scalability

The system can support additional tanks, buildings, and sensor devices. Sensor ingestion and alert services can be scaled independently when the number of readings increases.

5.2 Security

The platform uses secure authentication, role-based permissions, encrypted communication, and controlled device access. Activity logs support accountability and traceability.

5.3 Performance

Efficient ingestion, indexed database queries, and asynchronous alerts help process frequent sensor readings without delaying dashboard updates.

5.4 Reliability

Validation rules, retry mechanisms, local buffering, and backups help prevent lost readings during network failures. Fault alerts identify disconnected or malfunctioning sensors.

5.5 Maintainability

The layered, modular design separates dashboards, application functions, sensor ingestion, analytics, and storage so individual parts can be tested or replaced with limited impact.

5.6 Availability

Service monitoring, database backups, and recovery procedures improve availability. Redundant communication paths or deployments can be added for larger installations.

6. Justification of Architectural Decisions

6.1 Architectural Justification

The proposed Real-Time Water Management System uses a Hybrid Architecture combining Layered Architecture and Microservices. The approach organizes responsibilities into layers while allowing high-volume sensor processing and alerting to operate independently when needed.

The architecture is suitable because:

- Modularity: Each layer performs clearly defined water-monitoring functions.
- Scalability: Sensor ingestion and alerts can support more devices independently.
- Security: Authentication and encrypted communication protect access and data.
- Maintainability: Sensor and application modules can be updated separately.
- Extensibility: Additional sensor types and forecasting services can be integrated.

6.2 Architectural Trade-offs

Although the hybrid architecture offers several benefits, it also has some limitations:

- Implementation complexity: Multiple services require configuration and coordination.
- Communication overhead: Network communication can delay incoming readings.
- Deployment effort: Gateways and distributed services need monitoring and support.
- Security management: Devices, APIs, and databases must be protected.

For the initial academic prototype, the solution can use one modular backend connected to a small sensor network. Services may be separated as deployment size and measurement volume grow.

6.3 Future Enhancements

The architecture can be improved in the future by introducing:

1. AI-based prediction of water demand and abnormal consumption.
2. Integration with additional water quality and pressure sensors.
3. Automatic pump control with appropriate safety safeguards.
4. Interactive maps showing tanks, pipelines, and leakage alerts.
5. Cloud-based monitoring for multiple buildings and facilities.
6. Improved maintenance scheduling and long-term water usage forecasting.

7. Conclusion

The Real-Time Water Management System uses a hybrid architecture combining Layered Architecture and Microservices to enable reliable monitoring of water levels, consumption, flow, and pressure. It integrates IoT sensor collection, processing, analytics, alerting, secure storage, and dashboard visualization.

The proposed design improves scalability, security, maintainability, and timely detection of leakage or overflow. It helps residents and facility managers respond to problems and make informed water management decisions. The architecture provides a flexible foundation for future automation and expansion.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

	organized, professional report.			

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25